algorithms 2 homework 2

mahmood

April 19, 2023

Question 1. improve the algorithm DIJKSTRA so that it knows for every node v how many edges are in one of the shortest paths from the root node to v
show the correctness of the algorithm and its time complexity

Input:

1. a graph $G = (V, E, w)$, the graph G is weighted non-negatively ($w(e) \geq 0, e \in E$)
2. a vertex $s \in V$ (the origin)

Output: for every vertex $v \in V$ three variables, $d[v], l[v], p[v]$ with the following restrictions

1. $d[v] = \delta(s, v)$
2. $l[v]$ is the length of the shortest path from s to v
3. $p[v]$ is the second-to-last vertex in the minimal path from s to v

```
DIJKSTRA(G,s):
    DIJ-INIT(G,s)
    DIJ-OP(G,s)
DIJ-INIT(G,s):
    d[s] ← 0
    l[s] ← 0
    p[s] ← Null
    for v ∈ V − {s} do
        d[v] ← ∞
        l[v] ← ∞
        p[v] ← Null
DIJ-OP(G,s):
    PQ ← V  // priority queue
    while PQ ≠ ∅ do
        u ← PQ.extractMin()
        for v ∈ adj[u] do
            RELAX(u,v)
    return d, p
RELAX(u,v):
    if d[v] > d[u] + w(u, v) then
        d[v] ← d[u] + w(u, v)
        l[v] ← l[u] + 1
        p[v] ← u
```

Answer 1.0.1.

the time complexity is $O(|E| \log |V|)$

Question 2. improve the algorithm BFS so that it knows for every node v how many short paths there is from the root node to v
show the correctness of the algorithm and its time complexity

Answer 2.0.1.

```
Input: a graph G = (V, E) and a vertex s ∈ V
Output: an array of distances, such that for v ∈ V we have
        d[v] = δ(s, v), an array of vertices, such that for v ∈ V, p[v] is
        the second-to-last vertex in a minimal path from s to v, an
        array of integers, such that n[v] equals the number of
        minimal paths from s to v
BFS(G,s):
    BFS-INIT(G,s)
    while Q ≠ ∅ do
        u ← deque(Q)
        for v ∈ adj[u] do
            if d[v] = ∞ then
                d[v] ← d[u] + 1
                n[s] ← 1
                p[v] ← u
                enque(v)
            else if d[v] = d[u] + 1 then
                n[s] ← n[s] + 1
BFS-INIT(V,E,s):
    d[s] ← 0
    n[s] ← 0
    p[s] ← null
    for v ∈ V − {s} do
        d[v] ← ∞
        n[v] ← 0
        p[v] ← null
    enque(s)
```

the time complexity of the algorithm is still $\Theta(|E| + |V|)$

Question 3. given a directed and non-negatively weighted graph $G = (V, E, W)$ that represents a system of roads and junctions and the distances between them. given a set of junctions $F \subseteq V$ that contain gas stations. a car can carry fuel that allows it to travel K kilometers. given a starting junction u and a destination junction v, propose an algorithm that answers the question: can the car travel from the source junction to the destination junction (assume that the car starts with a full fuel container).

Answer 3.0.1. we use a modified version of dijkstra's algorithm

```
CAN-CAR-TRAVEL(G,F,K,u,v):
    DIJKSTRA(G,u,F,K)
    return d[v] ≥ 0
DIJKSTRA(G,s,F,K):
    DIJ-INIT(G,s)
    DIJ-OP(G,s,F,K)
DIJ-INIT(G,s):
    d[s] ← 0
    p[s] ← Null
    for v ∈ V − {s} do
        d[v] ← ∞
        p[v] ← Null
DIJ-OP(G,s,F,K):
    PQ ← V  // priority queue
    while PQ ≠ ∅ do
        u ← PQ.extractMin()  // vertex with minimal d value
        for v ∈ adj[u] do
            RELAX(G,u,v,F,K)
    return d, p
RELAX(G,u,v,F,K):
    if d[v] > d[u] + W(u,v) and d[u] + W(u,v) ≤ k then
        if v ∉ F then
            d[v] ← d[u] + W(u,v)
        else
            d[v] ← 0
            p[v] ← u
```

Question 4.

Subquestion 4.1. show an example of a non-weighted graph in which a distance tree relative to a specific vertex isnt the same for another vertex

Answer 4.1.1. pick a triangle, connect a vertex to one of its edges, this would result in the other 2 vertices having a distance of 2 from that new vertex yet the vertex we connected the new vertex to only has a distance of 1 from all the other vertices including the new one so the distance trees must differ

Subquestion 4.2. prove that if a specific spanning tree of a non-weighted graph is the distance tree relative to every vertex in the graph then the graph

is a tree

Answer 4.2.1. i couldnt solve this

Question 5. given a directed graph $G = (V, E)$, and given $s, t \in V, e_1 \in E$, propose an efficient algorithm that finds the shortest path that goes from s to t and goes through the edge e_1
explain why the algorithm works correctly, analyze the complexity of the algorithm

Answer 5.0.1. couldnt solve this either